

TL;DR — Java String Concept

- A **String** is a sequence of characters in Java.
- Strings are **immutable** → cannot be changed after creation.
- Stored in **Heap Memory**, with optimization through the **String Pool**.
- String literals reuse memory:

```
Java
String a = "Hi";
String b = "Hi"; // same memory reference
```

- `new String()` creates a separate object in Heap.
- `==` compares **memory/reference**.
- `.equals()` compares **actual text/content**.
- Common methods:
 - `length()`
 - `substring()`
 - `contains()`
 - `replace()`
 - `split()`
- Use **StringBuilder** for frequent string modifications for better performance.

One-line Summary

Java Strings are immutable objects optimized through the String Pool to save memory and improve performance.



ELI20 — String Concept in Java

A **String** in Java is simply text data.

Example:

```
Java
String name = "John";
```

Here, "John" is a String.

Important Things About Strings

1. Strings are Objects

In Java, Strings are not primitive data types like `int` or `char`.

They are objects created from the `String` class.

```
Java
String city = "Chennai";
```

2. Strings are Immutable

Immutable means:

+

Ask anything



2. Strings are Immutable

Immutable means:

Once a String is created, its value cannot be changed.

Example:

```
Java
String s = "Hello";
s = s + " World";
```

Java does not modify the original "Hello" string.

Instead:

- Old object: "Hello"
- New object created: "Hello World"

This improves:

- Security
- Performance
- Thread safety

3. String Pool (Memory Optimization)

Java stores commonly used string literals in a special memory area called the **String Pool**.

Example:

```
Java
```

3. String Pool (Memory Optimization)

Java stores commonly used string literals in a special memory area called the **String Pool**.

Example:

```
Java
String a = "Java";
String b = "Java";
```

Here:

- Only one "Java" object is created.
- Both `a` and `b` point to the same memory location.

This saves memory.

4. `new String()` Creates Extra Object

```
Java
String x = new String("Java");
```

This creates:

- One object in String Pool
- One separate object in Heap memory

So it uses more memory than normal string literals.

5. Comparing Strings

==

Checks whether both references point to the same object.

.equals()

Checks whether the actual text/content is same.

Example:

```
Java
String a = "Hi!";
String b = "Hi!";

a == b      // true
a.equals(b)  // true
```

But:

```
Java
String x = new String("Hi");
String y = new String("Hi");

x == y      // false
x.equals(y)  // true
```

Java

6. Common String Methods

```
Java
length()
substring()
contains()
replace()
split()
toUpperCase()
trim()
```

Example:

```
Java
String s = "Java";

s.length();      // 4
s.toUpperCase(); // JAVA
```

7. StringBuilder vs String

Since Strings are immutable, repeated modifications create many objects.

For heavy modifications, use:

```
Java
StringBuilder sb = new StringBuilder();
```



7. StringBuilder vs String

Since Strings are immutable, repeated modifications create many objects.

For heavy modifications, use:

Java

```
StringBuilder sb = new StringBuilder();
```



Better performance for:

- loops
- concatenation
- dynamic text building

Simple Real-world Analogy

Think of String like a printed paper.

If you want to change one word:

- You cannot edit the same paper.
- Java creates a new paper with updated content.

That is immutability.

